

XRPL Validator — Daily Engineering Report • 2026-09-17

Compiled retrospectively on 2026-09-23 from the git history. Scope: XRPL work only — the git history of [xrpl-validator](#) and the XRPL Rust SDK, the XRP Ledger itself (amendments) and XRPL research.

Summary

- 27 commits (xrpl-validator 27)
- 2 engine findings fixed (F291–F292)
- 3 test / vector commits
- Amendment **BatchV1_1** held majority (since 2026-09-15 14:06 UTC); activation due ~2026-09-29 14:06 UTC

Engine findings fixed

- **fix(xrpl-ledger): F291 — a crossing's remainder is re-derived from the sorted fold before `done` is believed** (5422429 , xrpl-validator) rippled recomputes remainingOut at every iteration boundary as `outReq - sum(savedOuts)` over the ASCENDING multiset (StrandFlow.h:791); the running chain is never consulted. Ten consecutive tecKILLED receipts (107044846-857, rGH4WSUU's Fill-or-Kill RLUSD buys) had four iterations whose chain decrements to exactly zero while the sorted 16-digit fold is 45.664757999999999 against 45.664758 wanted: a 1e-14 crumb. ...
- **fix(xrpl-ledger): F292 — under FlowSortStrands a strand behind the iteration's winner is not flowed, so it reaps nothing** (7bd0c74 , xrpl-validator) rippled flows an iteration's admitted strands in bound order and breaks at the first that succeeds and passes limitQuality (StrandFlow.h: `best.emplace(...); pushRemainingCurToNext(strandIndex + 1); break;`). The strands behind the winner are not flowed that iteration: no rev pass, no offer stream, nothing reaped.

Engineering changes

- **feat(xrpl-ledger): TxFields::from_json, inner_batch flag and the Batch tem codes** (46825c1 , xrpl-validator) Add `TxFields::inner_batch: bool` plus `fee_missing()` (waived for batch inners) and `from_json()`, moved from xrpl-node's `native_apply::build_txfields` (now a one-line delegate). `from_json` decodes the Account field via `tx::offer::decode20`, which accepts the engine's hex account dialect as well as base58 — `build_txfields`'s original `decode_address` call was base58-only and could not read the hex Account strings the ...
- **feat(xrpl-ledger): the fee-zero preflight gate is `fee_missing()`, waived for batch inners** (6b5a309 , xrpl-validator)
- **feat(xrpl-ledger): apply_on_sandbox — the tx pipeline on an existing sandbox with rollback** (99c4e14 , xrpl-validator) Mirrors `xrpl_node::native_apply::native_apply_one` but runs on a caller-supplied Sandbox instead of allocating a fresh one, restoring the entry snapshot on every non-applied path and keeping only the common changes on a claimed tec. Task 4 will call this per inner transaction of a Batch.

- **feat(xrpl-ledger): BatchTransactor — preflight, signer set, fee formula, and the four-mode inner loop** (20af7bd , xrpl-validator)
- **fix(xrpl-ledger): BatchTransactor — rippled 3.3.0 result codes, single-inner rejection, signer cap, cleared inner results** (05be0f5 , xrpl-validator)
- **feat(xrpl-node): the native shadow skips Batch inner entries and attributes them to their outer** (bd7b2c3 , xrpl-validator) rippled records every Batch (BatchV1_1) inner transaction as its own ledger entry — own hash, own metadata carrying ParentBatchID, the indices right after its outer — but APPLIES it inside the outer's application. Our BatchTransactor::do_apply does the same, so the shadow's replay must skip the inner entries and compare the outer's mutation set against the union of the outer's and its inners' AffectedNodes.
- **fix(xrpl-ledger): BatchTransactor decodes base58 and hex accounts alike (decode20)** (07e7a11 , xrpl-validator) decode_account accepted only the 40-hex-char dialect, on the assumption that native_apply::hexify_addresses had already rewritten every r-address in the transaction. It has not: that pass runs on ledger-state pre-images, and TxFields::from_json only rewrites the outer's own top-level account fields — nested RawTransaction/BatchSigner objects keep whatever dialect the feed sent. Every real Batch therefore ...
- **feat(xrpl-ledger): per-inner threading for Batch — touched-key tracking and stamp_batch_threading** (4c116ab , xrpl-validator) rippled applies every Batch inner as its own transaction (applyBatchTransactions -> apply(..., inner, tapBATCH)), so each ledger object an inner touched threads the INNER's transaction id, while the outer's own changes (its fee and sequence) thread the outer's. The last toucher wins. Our engine runs the inners inside the outer's do_apply and the callers stamped the whole mutation set with the outer's hash, ...
- **fix(xrpl-node): batch_vector harness warning; threadOwners coverage and a fallback receipt for the batch stamp** (c13f4c8 , xrpl-validator) assert_expect took a &LedgerState it never read, which the test target reported as unused variable: state — the parameter is gone from the signature and both call sites (the earlier clippy sweep never linted the file: it exits on the pre-existing errors in xrpl-ledger before reaching the node's test targets).
- **feat(xrpl-node): Batch attribution and per-inner threading in the replay tools' native legs** (90453a6 , xrpl-validator) state_replay and differential_probe iterate a ledger's entries the same way the native shadow does, and both were still applying Batch INNER entries standalone — which the engine rejects out of hand (tfInnerBatchTxn with Fee "0") — and stamping the outer's mutations with the outer's hash alone. Every Batch ledger therefore read as a divergence on every inner and a threading byte-diff on the outer.
- **fix(xrpl-node): differential_probe — guard the inner-result zip and print inner_ter in the text report** (e5fbfb0 , xrpl-validator) The kept inner ids are the ones the FIXTURE carries; the per-inner results are the inners the engine ATTEMPTED. Pairing them by index is meaningful only while the two lists agree — one hole shifts every later inner onto its neighbour's result and reports a confident, wrong verdict. The per-index comparison is now guarded by a length check; on disagreement it withholds the inner verdicts, prints one BATCH-INNER ...
- **fix(xrpl-ledger): Batch — fresh deferred-credits table per inner; kDisabledTxTypes; fee-formula and initiator notes** (2799e6d , xrpl-validator)
- **fix(xrpl-node): Batch inner pairing by id across the shadow and replay tools; inner TER counter; orphan const** (b52cb8e , xrpl-validator)
- **fix(ledger): OracleSet create writes Flags: 0 (finding 293)** (9934305 , xrpl-validator) sfFlags is soeREQUIRED on the Oracle ledger format, so a created Oracle carries Flags: 0 right after its type. The create

path built the object without it and serialized five bytes short (#107052956 B1EB9B20D12E, 172 bytes against mainnet's 177, the diff starting at byte 3) — the same shape as finding 187 on DID. New oracle_vector suite pins the object.

- **fix(ledger): own offers behind an issuer hop keep their owner credit; full delivery rests the line on pre + Amount (findings 294, 295)** (cec2d5a , xrpl-validator) #107052630 32386DDEB6B8: rUnRkdr pays 117.732708 USDT with FIL through the explicit issuer hop [rsL5Y] and the XRP bridge; strand 1 crosses the sender's OWN two FIL/USDT offers.
- **fix(ledger): reverse-pass dead-offer reaps survive the sizing snapshot (finding 296)** (45e8803 , xrpl-validator) #107009438 2877CBCC88C9: a deliver-max partial payment of 166716 drops for RLUSD through USDC.axl. rippled's reverse pass, asked for everything, walks the whole USDC.axl/RLUSD book: past the one funded tip it steps over rDeXHa's three unfunded offers and marks them ofrsToRm; the forward pass buys 0.2129 RLUSD from the tip alone, and the driver still deletes the three (plus book pages, owner page, OwnerCount) after ...
- **fix(ledger): the pool stands aside on its RAW quality, not the fee-inclusive spot (finding 297)** (1e5e7b8 , xrpl-validator) #107056200 BD9C7473B84F: rhTsmUJ's 6 XRP partial self-payment into RVR meets rMBPaL7's fresh offer at the tip and a 0.506% pool. rippled's AMMLiquidity::getOffer (AMMLiquidity.cpp:181-183) stands the pool aside only when Quality{balances} — the raw pool quality, no trading fee — is not strictly better than the tip or sits within 1e-7 of it; the fee enters only in the anchored offer changeSpotPriceQuality ...
- **fix(ledger): a multi-path pool is gated by its own book's tip, not only the rival strand (finding 298)** (a1aecb0 , xrpl-validator) #107060755 F43C3C4DA037: rsdsSA7's tfSell offer, 852332.25 ASC for 599530.50 PLR, meets a direct ASC/PLR pool and a two-pool XRP bridge — multi-path, Fibonacci offers. rippled takes five direct-pool slices (25.645 ASC), then AMMLiquidity::getOffer (AMMLiquidity.cpp:181-196) stands the pool aside because its RAW quality (1.40353) no longer beats its own book's tip — the taker's own offer 4CF7660E at 1.401935, still ...

Tests and campaigns

- **test(xrpl-ledger): apply_on_sandbox — exercise the do_apply rollback branch; drop an unused test import** (f5f5ab6 , xrpl-validator)
- **test(xrpl-node): devnet Batch vectors — merged outer+inner bundles, and the merge script** (73097b1 , xrpl-validator)
- **test(xrpl-node): batch_vector — byte-exact devnet Batch vectors through the native transactor (with per-inner threading in the harness and the shadow)** (a57a648 , xrpl-validator)

Docs and operations

- **docs: Batch leg B implementation plan (native BatchTransactor, shadow attribution, devnet vectors)** (36dbd07 , xrpl-validator)
- **docs: Batch spec status — leg B implemented and verified on devnet** (c57e2d4 , xrpl-validator)
- **docs: Batch spec — kDisabledTxTypes correction and honest status; merge script docstring** (1cb22ad , xrpl-validator)
- **build(ff): port the libxrpl shim to xrpld 3.4.0** (13d08f1 , xrpl-validator)

Amendment watch

Read from the XRP Ledger: the Amendments object as of the last ledger of the day, and the day's EnableAmendment pseudo-transactions.

Amendment	Majority since	Activation due
BatchV1_1	2026-09-15 14:06 UTC	2026-09-29 14:06 UTC

Amendments enabled on mainnet: 94.

XRPL research

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **Permission-Gated SingleAssetVault Deposits** (compliance) Add an optional sfDomainID SField to the Vault ledger object (ItVAULT), settable only at VaultCreate, referencing an existing PermissionedDomain. When
- **Pseudo-Account Custody Sub-Ledgers** (custody) Add a SubAccountCreate transaction that mints a pseudo-account per the XLS-64 keyless-account derivation (keylet::subAccount(parentID, seq)), recorded

Appendix — commits

Time	Repo	Commit	Subject
08:00	xrpl-validator	5422429	fix(xrpl-ledger): F291 — a crossing's remainder is re-derived from the sorted fold before <code>done</code> is believed
08:31	xrpl-validator	7bd0c74	fix(xrpl-ledger): F292 — under FlowSortStrands a strand behind the iteration's winner is not flowed, so it reaps nothing
08:45	xrpl-validator	36dbd07	docs: Batch leg B implementation plan (native BatchTransactor, shadow attribution, devnet vectors)
09:38	xrpl-validator	46825c1	feat(xrpl-ledger): TxFields::from_json, inner_batch flag and the Batch tem codes
09:48	xrpl-validator	6b5a309	feat(xrpl-ledger): the fee-zero preflight gate is fee_missing(), waived for batch inners
09:57	xrpl-validator	99c4e14	feat(xrpl-ledger): apply_on_sandbox — the tx pipeline on an existing sandbox with rollback
10:06	xrpl-validator	f5f5ab6	test(xrpl-ledger): apply_on_sandbox — exercise the do_apply rollback branch; drop an unused test import
10:18	xrpl-validator	20af7bd	feat(xrpl-ledger): BatchTransactor — preflight, signer set, fee formula, and the four-mode inner loop
10:40	xrpl-validator	05be0f5	fix(xrpl-ledger): BatchTransactor — rippled 3.3.0 result codes, single-inner rejection, signer cap, cleared inner results
11:01	xrpl-validator	bd7b2c3	feat(xrpl-node): the native shadow skips Batch inner entries and attributes them to their outer
11:16	xrpl-validator	73097b1	test(xrpl-node): devnet Batch vectors — merged outer+inner bundles, and the merge script

Time	Repo	Commit	Subject
11:40	xrpl-validator	07e7a11	fix(xrpl-ledger): BatchTransactor decodes base58 and hex accounts alike (decode20)
11:49	xrpl-validator	4c116ab	feat(xrpl-ledger): per-inner threading for Batch — touched-key tracking and stamp_batch_threading
11:49	xrpl-validator	a57a648	test(xrpl-node): batch_vector — byte-exact devnet Batch vectors through the native transactor (with per-inner threading in the harness and the shadow)
12:05	xrpl-validator	c13f4c8	fix(xrpl-node): batch_vector harness warning; threadOwners coverage and a fallback receipt for the batch stamp
12:30	xrpl-validator	90453a6	feat(xrpl-node): Batch attribution and per-inner threading in the replay tools' native legs
12:42	xrpl-validator	e5fbfb0	fix(xrpl-node): differential_probe — guard the inner-result zip and print inner_ter in the text report
12:52	xrpl-validator	c57e2d4	docs: Batch spec status — leg B implemented and verified on devnet
13:20	xrpl-validator	2799e6d	fix(xrpl-ledger): Batch — fresh deferred-credits table per inner; kDisabledTxTypes; fee-formula and initiator notes
13:28	xrpl-validator	b52cb8e	fix(xrpl-node): Batch inner pairing by id across the shadow and replay tools; inner TER counter; orphan const
13:29	xrpl-validator	1cb22ad	docs: Batch spec — kDisabledTxTypes correction and honest status; merge script docstring
16:15	xrpl-validator	9934305	fix(ledger): OracleSet create writes Flags: 0 (finding 293)
16:32	xrpl-validator	cec2d5a	fix(ledger): own offers behind an issuer hop keep their owner credit; full delivery rests the line on pre + Amount (findings 294, 295)
18:18	xrpl-validator	45e8803	fix(ledger): reverse-pass dead-offer reaps survive the sizing snapshot (finding 296)
20:01	xrpl-validator	1e5e7b8	fix(ledger): the pool stands aside on its RAW quality, not the fee-inclusive spot (finding 297)
21:54	xrpl-validator	13d08f1	build(ffl): port the libxrpl shim to xrpld 3.4.0
23:47	xrpl-validator	a1aecb0	fix(ledger): a multi-path pool is gated by its own book's tip, not only the rival strand (finding 298)